cam_capture.v, explained part by part

Captures one camera frame and converts it into 320 x 320 colour pixels, DE2-115 live JPEG compression project. Prepared by Dev Vijay for the project review.

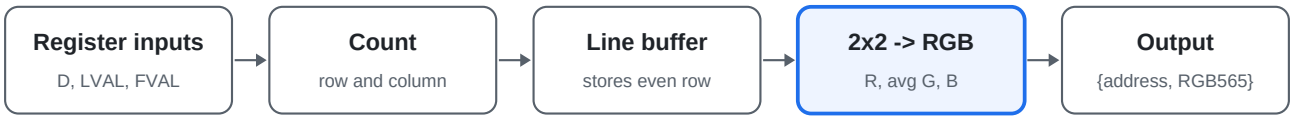
1. What the module does

The camera sends a 640 x 640 picture one pixel per clock on its own clock, **PIXCLK**. Each camera pixel is only **one colour**: the sensor has a colour filter in a repeating 2 x 2 pattern called a **Bayer pattern**:

```
     column:  even   odd
even row:     G      R
 odd row:     B      G
```

cam_capture waits for the start of a frame, then for every 2 x 2 group makes **one full-colour pixel**: R = the red dot, B = the blue dot, G = the average of the two greens. 640 x 640 dots become **320 x 320 RGB pixels**, exactly the size the compression engine uses. Each finished pixel is sent out with its memory address to the FIFO.

Data path inside cam_capture (all on the camera clock PIXCLK)



only between two frame starts after a capture request (one whole frame)

Camera signals used

Signal	Meaning
PIXCLK	pixel clock from the camera; one pixel per rising edge
D[11:0]	pixel value, 12 bits (0..4095); we keep the top 8 bits
LVAL	line valid: 1 while a row of pixels is being sent
FVAL	frame valid: 1 for the whole frame, 0 between frames

2. The code, part by part

Part A - Ports (lines 24-34)

Lines 24-34

```
module cam_capture (
    input  wire        pclk,          // camera pixel clock
    input  wire [11:0] d,             // camera pixel data (12 bits)
    input  wire        lval,          // line valid
    input  wire        fval,          // frame valid
    input  wire [1:0]  bayer_phase,   // SW3 SW2: which corner of 2x2 is red
    input  wire        cap_req,       // from the 25 MHz side (not synchronised yet)
    output reg         cap_done,      // to the 25 MHz side
    output reg         wen,           // 1 = wdata holds a new pixel
    output reg  [32:0] wdata          // {address[16:0], RGB565[15:0]}
);
```

- **bayer_phase** (SW3, SW2) tells which corner of the 2 x 2 group is red, because the pattern can shift by one row or column after mirroring.
- **cap_req / cap_done** are the handshake with the 25 MHz control logic.
- **wdata** is 33 bits: a 17-bit address (0..102,399) plus a 16-bit RGB565 pixel.

Part B - Register the inputs and synchronise the request (lines 36-49)

Lines 36-49

```
    // ---------- register the camera inputs ----------
    reg [7:0] d_r;                  // top 8 of the 12 bits are enough for display
    reg       lval_r, fval_r, lval_d, fval_d;
    reg       req_1, req_2;         // 2-stage synchroniser for cap_req

    always @(posedge pclk) begin
        d_r    <= d[11:4];
        lval_r <= lval;
        fval_r <= fval;
        lval_d <= lval_r;
        fval_d <= fval_r;
        req_1  <= cap_req;
        req_2  <= req_1;
    end
```

- Every camera input is copied into a flip-flop on PIXCLK first. This gives clean, stable values for the logic after it.
- **d[11:4]** keeps the top 8 of the 12 bits: enough for display and the same size as the rest of the design.
- **lval_d, fval_d** are the values one clock earlier; comparing old and new detects edges (start/end of a line or frame).
- **req_1, req_2** form a **2-flip-flop synchroniser**: cap_req comes from the 25 MHz clock, which is unrelated to PIXCLK, so it is passed through two flip-flops to avoid metastability (an undefined value when a signal changes exactly at a clock edge).

Part C - Row and column counters (lines 51-60)

Lines 51-60

```
    // ---------- row / column counters ----------
    reg [10:0] col = 11'd0;
    reg [10:0] row = 11'd0;
    always @(posedge pclk) begin
        if (lval_r) col <= col + 11'd1;
        else        col <= 11'd0;

        if (!fval_r)                row <= 11'd0;
        else if (lval_d && !lval_r) row <= row + 11'd1;   // end of a line
    end
```

- **col** counts pixels while LVAL = 1 and returns to 0 between lines.
- **row** goes up by 1 at the end of each line (LVAL falls: old value 1, new value 0) and returns to 0 between frames.
- 11 bits allow counts up to 2047, more than the 640 we need.

Part D - Capture exactly one whole frame (lines 62-77)

Lines 62-77

```
    // ---------- capture exactly one whole frame ----------
    reg active = 1'b0;
    initial cap_done = 1'b0;
    always @(posedge pclk) begin
        if (!req_2) begin
            active   <= 1'b0;
            cap_done <= 1'b0;
        end else if (!cap_done) begin
            if (fval_r && !fval_d)
                active <= 1'b1;                          // frame starts
            else if (!fval_r && fval_d && active) begin
                active   <= 1'b0;                        // frame ends
                cap_done <= 1'b1;
            end
        end
    end
```

- When the request arrives (req_2 = 1), **active** waits for the **rising edge of FVAL**, the start of a new frame. This guarantees we never capture half a frame.

- At the **falling edge of FVAL** (frame finished) active turns off and **cap_done** = 1.
- When the control logic lowers cap_req, cap_done returns to 0, ready for the next picture. This request-done-release-clear sequence is a **4-phase handshake**, safe across two unrelated clocks.

Part E - Line buffer (lines 79-91)

Lines 79-91

```
    // ---------- line buffer: top row of each 2x2 group ----------
    reg [15:0] lb [0:511];           // {top-left, top-right} for each column pair
    reg [15:0] lb_q;
    reg [7:0]  prev;                 // pixel to the left of the current one
    wire [8:0] pair = col[9:1];

    always @(posedge pclk) begin
        if (lval_r && !row[0] && col[0])
            lb[pair] <= {prev, d_r};                    // even row: store the pair
        lb_q <= lb[pair];                               // read it back on the odd row
        if (lval_r && !col[0])
            prev <= d_r;
    end
```

- A 2 x 2 group spans two rows, but the camera sends one row at a time. The even (top) row must be remembered until the odd (bottom) row arrives.
- **prev** remembers the pixel to the left (the even column of each pair).
- On an **even row** at each odd column, the pair {left, current} is stored at **lb[col/2]**. lb has 512 x 16 bits, one block RAM.
- On the **odd row**, **lb_q** reads the stored pair back. The read is clocked, so the pair read at the even column is ready one clock later, exactly at the odd column when all four dots are available.

Part F - Four dots to one RGB pixel (lines 93-109)

Lines 93-109

```
    // ---------- 2x2 group -> R G B ----------
    wire [7:0] p00 = lb_q[15:8];      // top-left
    wire [7:0] p01 = lb_q[7:0];       // top-right
    wire [7:0] p10 = prev;            // bottom-left
    wire [7:0] p11 = d_r;             // bottom-right (arriving now)
    wire [8:0] g_a = p00 + p11;
    wire [8:0] g_b = p01 + p10;

    reg [7:0] R, G, B;
    always @(*) begin
        case (bayer_phase)
            2'd0: begin R = p01; B = p10; G = g_a[8:1]; end    // G R / B G
            2'd1: begin R = p00; B = p11; G = g_b[8:1]; end    // R G / G B
            2'd2: begin R = p11; B = p00; G = g_b[8:1]; end    // B G / G R
            default: begin R = p10; B = p01; G = g_a[8:1]; end // G B / R G
        endcase
    end
```

Name	Dot	Comes from
p00	top-left	line buffer, high byte
p01	top-right	line buffer, low byte
p10	bottom-left	prev (the pixel one clock ago)
p11	bottom-right	d_r (the pixel arriving now)

- The two greens are on a diagonal: either (p00, p11) or (p01, p10). **g_a** and **g_b** add each pair in 9 bits (so 255 + 255 does not overflow), and **[8:1]** divides by 2.
- The **case** on bayer_phase covers all four possible arrangements, so the right colours can be chosen with SW3, SW2 without recompiling.

Part G - Output pixel (lines 111-124)

Lines 111-124

```
    // ---------- output pixel (row/2, col/2) ----------
    wire [9:0]  orow = row[10:1];
    wire [9:0]  ocol = col[10:1];
    wire [16:0] oaddr = ({7'd0, orow} << 8) + ({7'd0, orow} << 6) + {7'd0, ocol};   // orow*320+ocol

    always @(posedge pclk) begin
        wen <= 1'b0;
        if (active && lval_r && row[0] && col[0] && (orow < 10'd320) && (ocol < 10'd320)) begin
            wen   <= 1'b1;
            wdata <= {oaddr, R[7:3], G[7:2], B[7:3]};
        end
    end

endmodule
```

- A pixel is produced only when **active**, during a line, on an **odd row and odd column**: the moment the last dot of a 2 x 2 group arrives. That is 1 output every 4 camera pixels on average.
- **orow = row / 2, ocol = col / 2** (dropping the lowest bit). Address = orow x 320 + ocol, with x 320 done as (orow x 256) + (orow x 64) using shifts.
- The check < 320 protects the memory if the camera ever sends a slightly larger frame.
- **RGB565**: the top 5 bits of red, 6 of green, 5 of blue = 16 bits, so the whole picture fits in on-chip memory.

Optional add-on: digital brightness (SW5, SW4)

If the brightness edit was added, the line **d_r <= d[11:4]** becomes **d_r <= d_gain**, where d_gain picks a lower group of 8 bits (d[10:3], d[9:2] or d[8:1]) to make the picture 2, 4 or 8 times brighter, and gives 255 if any higher bit is set (so bright areas stay white instead of wrapping round to dark).

3. Questions the professor may ask

Q: What is a Bayer pattern?

A colour filter over the sensor so each photosite sees only red, green or blue, in a repeating G R / B G pattern. Green appears twice because the eye is most sensitive to green.

Q: How do you get full colour from one-colour dots?

We combine each 2 x 2 group: R from the red dot, B from the blue dot, G as the average of the two greens. This is a simple form of demosaicing that also halves the size.

Q: Why do you need a line buffer?

The camera sends the top and bottom rows of a 2 x 2 group at different times. The top row is stored until the bottom row arrives.

Q: Why synchronise cap_req with two flip-flops?

It comes from a different clock. A signal changing exactly at a clock edge can leave a flip-flop in an undefined state for a short time; a second flip-flop gives it time to settle.

Q: How do you make sure a full frame is captured?

Capture starts only on the rising edge of FVAL and ends on its falling edge, so the frame is always complete.

Q: Why keep only 8 of the 12 bits?

The display and compression work in 8 bits per colour, and 8 bits keep memory small. The brightness add-on chooses which 8 bits to keep.

Q: What clock does this module use?

Only the camera's PIXCLK. The FIFO after it moves the pixels to the 25 MHz clock.